

1) What a strong microservices capstone should demonstrate

A good capstone is not just “many small services.” It should show that you understand:

- bounded contexts^{**}: each service owns a specific business capability
- independent deployment^{**}: services can evolve separately
- API-based communication^{**}: REST, sometimes async messaging
- data ownership: each service should ideally own its own database/schema
- fault tolerance: one service failure should not crash the whole system
- observability: logs, health checks, monitoring
- security: authentication, authorization, API protection
- frontend integration: React app consuming backend APIs through a gateway

2) Recommended project structure

Frontend

Use ^{**}React^{**} for:

- * login/signup
- * dashboards
- * forms and workflows
- * admin panels
- * analytics pages

Backend choices

You can mix technologies to show polyglot microservices:

- Python: Flask/FastAPI for ML, analytics, reporting, automation-heavy services
- Node.js :Express/NestJS for real-time APIs, notification services, lightweight business APIs
- Spring Boot: strong choice for core enterprise services like user management, orders, payments, inventory

A strong hybrid stack could be:

- Spring Boot for main business services
- Node.js for notification/chat/realtime service
- Python FastAPI for recommendation, analytics, fraud/risk scoring, forecasting

3) Core architecture to include

A clean capstone architecture can look like this:

- React frontend**
- API Gateway**
- Auth Service**
- User Service**
- Business domain services** (3 to 6 services)
- Notification Service**
- Analytics/Reporting Service**
- Databases per service**
- optional: **Message broker** like RabbitMQ or Kafka
- Docker / Docker Compose**
- optional: Kubernetes for advanced deployment

Supporting components

Try to include at least some of these:

* health endpoints

* Swagger / OpenAPI docs

4) Typical microservices to include in any case study

Regardless of the domain, these services are commonly useful:

1. Authentication Service

Handles:

* signup/login

* JWT token generation

* password hashing

* role-based access control

2. User/Profile Service

Handles:

* profile data

* preferences

- * roles

- * account settings

3. Domain Service(s)

These are your business services, such as:

- * Order Service

- * Booking Service

- * Appointment Service

- * Product Service

- * Course Service

- * Case Management Service

4. Notification Service

Handles:

- * email

- * SMS

- * in-app notifications

- * event-based alerts

5. Analytics/Reporting Service

Handles:

- * reports

- * dashboards

- * usage trends

- * prediction/recommendation

6. API Gateway

Handles:

- * routing

- * auth validation

- * aggregation

- * rate limiting

5) Capstone implementation guideline

Phase 1: Problem definition

Choose a domain with:

- * clear actors

- * multiple workflows

- * separate business capabilities

- * realistic need for scaling or modularization

Define:

- * who uses the system

- * what pain point it solves

- * why microservices are better than a monolith here

Phase 2: Service decomposition

Break the system by business capability, not by technical layer.

Bad decomposition:

- * controller service

- * database service

- * validation service

Good decomposition:

- * auth service

- * booking service

- * payment service

- * notification service

- * reporting service

Phase 3: Communication design

Synchronous

Use REST APIs for:

- * immediate request/response workflows

- * user login

- * fetching data

- * creating entities

Asynchronous

Use message queues/events for:

- * sending notifications

- * updating analytics

- * triggering background jobs

- * handling eventual consistency

For a capstone, a good balance is:

- * REST for core flows

- * RabbitMQ/Kafka for one or two event-driven flows[optional]

Phase 4: Database design

Each service should own its data.

Example:

- * Auth Service → users/auth DB
- * Product Service → product DB
- * Order Service → order DB
- * Analytics Service → reporting DB

If full separate databases are too heavy, at least use separate schemas/tables and clearly document service ownership.

Phase 5: Frontend integration

React frontend should:

- * authenticate via Auth Service
- * call the API Gateway instead of every service directly
- * support role-based views
- * show dashboards and business workflows
- * handle error states properly

Add:

- * loading states
- * protected routes
- * reusable API client
- * charts if applicable

Phase 6: DevOps and deployment

At minimum:

- * Dockerize every service

- * use Docker Compose for orchestration
- * provide README with startup instructions

For stronger impact:[Optional]

- * CI/CD pipeline
- * Kubernetes manifests
- * Nginx reverse proxy
- * centralized logs

Phase 7: Testing and documentation

Include:

- * unit tests
- * API tests
- * integration tests
- * architecture diagram
- * sequence diagram
- * ER diagram
- * API documentation
- * demo script

6) Evaluation rubric you can use

A very good capstone usually scores well on these dimensions:

Functional quality

- * all major flows work
- * clean UI
- * proper validation
- * role-based access

Architecture quality

- * clear service boundaries
- * API gateway usage
- * service-to-service communication
- * loose coupling

Engineering quality

- * Dockerized setup
- * logging
- * testing
- * documentation
- * clean Git structure

Innovation

- * hybrid tech stack
- * event-driven design
- * analytics/ML integration
- * scalability reasoning